Overcoming Monolithic Architecture Challenges With Microservices

1. Scalability

- In a microservices architecture, each service is separate and can be **scaled independently**

Monolithic Approach

- The monolithic approach may require scaling the entire application

```
High Seller Traffic
        ↓
Scale entire application
        ↓
User + Product + Inventory + Payment + Order + Seller
```

This can waste resources.

With microservices:

```
Seller Service
      ↓
High traffic
      ↓
Scale Seller Service only
```

The other services don't necessarily need to be scaled.

Black Friday Example

During a Black Friday sale, you might have:

```
Product Service    → High traffic
Payment Service    → High traffic
Inventory Service  → High traffic
Order Service      → High traffic
User Service       → Normal traffic
Support Service    → Normal traffic
```

You can selectively scale the services experiencing high demand:

```
Product      → ↑ Scale
Payment      → ↑ Scale
Inventory    → ↑ Scale
Order        → ↑ Scale

User         → No change
Support      → No change
```

2. Flexibility

- Microservices also provide **technology flexibility**
- Because services are **independently developed,** different services can potentially use **different technology stacks**

For example:

```
User Service
    → Technology A

Product Service
    → Technology B

Payment Service
    → Technology C

Inventory Service
    → Technology D
```

3. Independent Development and Deployment

With a monolith:

```
Change Inventory
      ↓
Build entire application
      ↓
Deploy entire application
```

With microservices:

```
Change Inventory
      ↓
Build Inventory Service
      ↓
Test Inventory Service
      ↓
Deploy Inventory Service
```

Other services don't necessarily need to be redeployed.

This supports:

- Faster development
- Faster releases
- Continuous development
- Continuous delivery
- Independent deployment

4. Independent Management

Each microservice can also be managed separately.

For example:

```
User Service       → Team A
Product Service    → Team B
Inventory Service  → Team C
Order Service      → Team D
Payment Service    → Team E
```

Each team can focus primarily on its own service.

This supports greater **team autonomy**.

5. Monolith Vs. Microservices

Area	Monolithic	Microservices
Scalability	Usually scale application as a whole	Scale individual services
Technology	Strong commitment to common stack	Services can potentially use different stacks
Codebase	Large, centralized	Smaller service-level codebases
Deployment	Usually entire application	Individual services can be deployed
Development	More interdependencies	Greater service-level independence
Team autonomy	More coordination required	Teams can have greater autonomy
Changes	Can affect/redeploy entire app	Can often be isolated to one service